Medical Computer Vision Lab

Scalable ML Experimentation with Argument Parsers: Lab Standard

Maintainer: Ashery Mbilinyi

Version 1.0 : October 16, 2026

Contents

1 Purpose **2**

2 Why Argument Parsers Enable Scalability **2**
2.1 Key Benefits . . . 2

3 Lab Policy: No Code Changes for Experiments **2**
3.1 Rule . . . 2
3.2 Allowed Ways to Configure Experiments . . . 2
3.3 Disallowed . . . 2

4 Standard Experiment Output Structure **3**

5 Core Arguments Required in All Training Scripts **3**
5.1 Experiment / Logging . . . 3
5.2 Dataset . . . 3
5.3 Model . . . 3
5.4 Optimization . . . 3
5.5 Reproducibility . . . 3
5.6 Hardware . . . 4
5.7 Resume / Checkpointing . . . 4
5.8 Config File Support (Mandatory) . . . 4

6 Standard CLI Usage Examples **4**
6.1 Baseline run . . . 4
6.2 Config-based run . . . 4
6.3 Config run with overrides . . . 4
6.4 Sweeps . . . 4

7 Recommended Implementation Template **4**
7.1 Full Reference Template . . . 4

8 Config File Example **8**

9 Checklist for Code Reviews **8**

10 Recommended Extensions (Optional) **9**

1 Purpose

This document defines our lab standard for writing scalable machine learning (ML) experiment code using argument parsers. The goal is to ensure our experiments are:

- **Reproducible**: every run can be reproduced exactly
- **Auditable**: every result is traceable to a configuration
- **Scalable**: easy to run hundreds of experiments on local machines, servers, and HPC clusters
- **Collaborative**: new students can run baselines without modifying code

2 Why Argument Parsers Enable Scalability

Argument parsers (e.g., Python `argparse`, Hydra, Typer) improve scalability by turning scripts into **configurable tools** rather than one-off code.

2.1 Key Benefits

- **Eliminates code editing for experimentation**.
  Parameters should not require editing the training script. Instead, experiments are configured from the command line.
- **Reproducibility at scale**.
  Each run can save its full configuration (`config.json`, `cmd.txt`), enabling exact reproduction months later.
- **Simple sweeps and automation**.
  Sweeps become trivial through bash loops, schedulers, or frameworks (SLURM, Ray Tune, W&B sweeps).
- **HPC and pipeline compatibility**.
  Cluster job submission is easiest when scripts take structured arguments and output to predictable directories.
- **Safer experiments**.
  Parsers support type-checking, required fields, and limited choices, reducing wasted GPU runs.

3 Lab Policy: No Code Changes for Experiments

3.1 Rule

Model experimentation must not require changing source code.

3.2 Allowed Ways to Configure Experiments

- CLI arguments (`--lr`, `--batch_size`, `--seed`, etc.)
- Config files (`--config configs/baseline.json`)

3.3 Disallowed

- Editing constants inside scripts (learning rate, batch size, epochs)
- Hardcoding dataset paths in code
- Changing the script for each experiment

4 Standard Experiment Output Structure

Every experiment run should create a unique run directory containing:

- `config.json` : resolved configuration used for the run
- `cmd.txt` : command used to launch the run
- `checkpoints/` : saved model checkpoints
- `logs/` or metrics files

Example:

```
runs/
  rsna_trauma_resnet50_lr0.0001_20260110_215000/
    config.json
    cmd.txt
    checkpoints/
      last.ckpt
      best.ckpt
```

5 Core Arguments Required in All Training Scripts

At minimum, every training script must support:

5.1 Experiment / Logging

- `--run_name` (auto-generated if empty)
- `--out_dir` (default: `runs`)

5.2 Dataset

- `--dataset` (choices must be enforced)
- `--data_dir`
- `--num_workers`

5.3 Model

- `--model` (choices enforced)
- `--pretrained`

5.4 Optimization

- `--epochs`
- `--batch_size`
- `--lr`
- `--weight_decay`
- `--optimizer` (choices enforced)
- `--scheduler` (choices enforced)

5.5 Reproducibility

- `--seed`
- `--deterministic`

5.6 Hardware

- `--device` (cpu/cuda/mps)
- `--amp`

5.7 Resume / Checkpointing

- `--resume` (checkpoint path)

5.8 Config File Support (Mandatory)

- `--config` (JSON/YAML config)

6 Standard CLI Usage Examples

6.1 Baseline run

```
python train.py --dataset rsna_trauma --model resnet50 --lr 1e-4 --seed 0 --amp
```

6.2 Config-based run

```
python train.py --config configs/baseline.json
```

6.3 Config run with overrides

```
python train.py --config configs/baseline.json --lr 5e-5 --seed 2
```

6.4 Sweeps

```
for seed in 0 1 2; do
  for lr in 1e-3 1e-4 1e-5; do
    python train.py \
      --dataset rsna_trauma \
      --model resnet50 \
      --lr $lr \
      --seed $seed \
      --amp
  done
done
```

7 Recommended Implementation Template

Below is the lab standard `argparse` training entrypoint template. Students may adapt it, but required fields and output structure must remain consistent.

7.1 Full Reference Template

```
#!/usr/bin/env python3
"""
train.py - scalable training entrypoint template

Usage:
  python train.py --dataset mimic_ecg --model resnet18 --lr 1e-4 --seed 0
  python train.py --config configs/baseline.json --batch_size 64
  python train.py --resume runs/exp_001/checkpoints/last.ckpt
```

```
"""

from __future__ import annotations

import argparse
import json
import random
import shutil
import sys
from dataclasses import asdict, dataclass
from datetime import datetime
from pathlib import Path
from typing import Any

import numpy as np

try:
    import torch
except ImportError:
    torch = None


def set_seed(seed: int, deterministic: bool = False) -> None:
    random.seed(seed)
    np.random.seed(seed)

    if torch is not None:
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)

        if deterministic:
            torch.backends.cudnn.deterministic = True
            torch.backends.cudnn.benchmark = False


@dataclass
class TrainConfig:
    # Experiment / logging
    run_name: str
    out_dir: str
    save_every: int
    log_every: int

    # Dataset
    dataset: str
    data_dir: str
    num_workers: int

    # Model
    model: str
    pretrained: bool

    # Optimization
    epochs: int
    batch_size: int
    lr: float
    weight_decay: float
    optimizer: str
    scheduler: str

    # Reproducibility
    seed: int
    deterministic: bool
```

```
    # Hardware
    device: str
    amp: bool

    # Resume
    resume: str | None


def timestamp() -> str:
    return datetime.now().strftime("%Y%m%d_%H%M%S")


def ensure_dir(path: str | Path) -> Path:
    path = Path(path)
    path.mkdir(parents=True, exist_ok=True)
    return path


def dump_json(path: Path, obj: dict[str, Any]) -> None:
    with open(path, "w", encoding="utf-8") as f:
        json.dump(obj, f, indent=2, sort_keys=True)


def load_json(path: str | Path) -> dict[str, Any]:
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)


def save_run_config(run_dir: Path, cfg: TrainConfig, argv: list[str]) -> None:
    dump_json(run_dir / "config.json", asdict(cfg))
    (run_dir / "cmd.txt").write_text("python " + " ".join(argv) + "\n", encoding
    ="utf-8")


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="train.py",
        description="Scalable training entrypoint",
        formatter_class=argparse.ArgumentDefaultsHelpFormatter,
    )

    g = parser.add_argument_group("Experiment / Logging")
    g.add_argument("--run_name", type=str, default="", help="Run name (auto if
    empty).")
    g.add_argument("--out_dir", type=str, default="runs", help="Base output
    directory.")
    g.add_argument("--save_every", type=int, default=1, help="Checkpoint every N
     epochs.")
    g.add_argument("--log_every", type=int, default=50, help="Log every N steps.
    ")

    g = parser.add_argument_group("Dataset")
    g.add_argument(
        "--dataset",
        type=str,
        default="dummy",
        choices=["dummy", "ptbxl", "mimic_ecg", "rsna_trauma"],
    )
    g.add_argument("--data_dir", type=str, default="data")
    g.add_argument("--num_workers", type=int, default=4)

    g = parser.add_argument_group("Model")
```

```
    g.add_argument(
        "--model",
        type=str,
        default="resnet18",
        choices=["resnet18", "resnet50", "unet", "vit_b16"],
    )
    g.add_argument("--pretrained", action="store_true")

    g = parser.add_argument_group("Optimization")
    g.add_argument("--epochs", type=int, default=50)
    g.add_argument("--batch_size", type=int, default=32)
    g.add_argument("--lr", type=float, default=1e-4)
    g.add_argument("--weight_decay", type=float, default=1e-4)
    g.add_argument("--optimizer", type=str, default="adamw", choices=["sgd", "
    adam", "adamw"])
    g.add_argument("--scheduler", type=str, default="cosine", choices=["none", "
    step", "cosine"])

    g = parser.add_argument_group("Reproducibility")
    g.add_argument("--seed", type=int, default=0)
    g.add_argument("--deterministic", action="store_true",
                   help="Force deterministic behavior (slower).")

    g = parser.add_argument_group("Hardware")
    g.add_argument("--device", type=str, default="cuda", choices=["cuda", "cpu",
     "mps"])
    g.add_argument("--amp", action="store_true", help="Enable mixed precision.")

    g = parser.add_argument_group("Resume")
    g.add_argument("--resume", type=str, default=None)

    parser.add_argument("--config", type=str, default=None,
                        help="Optional JSON config to load defaults from.")

    return parser


def parse_args_with_config(parser: argparse.ArgumentParser, argv: list[str]) ->
    argparse.Namespace:
    partial, _ = parser.parse_known_args(argv)

    if partial.config is not None:
        cfg = load_json(partial.config)
        parser.set_defaults(**cfg)

    args = parser.parse_args(argv)
    return args


def train(cfg: TrainConfig, run_dir: Path) -> None:
    print(json.dumps(asdict(cfg), indent=2))
    set_seed(cfg.seed, deterministic=cfg.deterministic)

    # TODO: implement actual training loop
    print(f"Training would run here. Outputs at: {run_dir}")


def main(argv: list[str]) -> int:
    parser = build_parser()
    args = parse_args_with_config(parser, argv)

    if args.run_name.strip() == "":
        args.run_name = f"{args.dataset}_{args.model}_lr{args.lr}_{timestamp()}"
```

```
    base_out = Path(args.out_dir)
    run_dir = ensure_dir(base_out / args.run_name)
    ensure_dir(run_dir / "checkpoints")

    if args.config is not None:
        shutil.copy(args.config, run_dir / "config_used.json")

    cfg = TrainConfig(
        run_name=args.run_name,
        out_dir=str(run_dir),
        save_every=args.save_every,
        log_every=args.log_every,
        dataset=args.dataset,
        data_dir=args.data_dir,
        num_workers=args.num_workers,
        model=args.model,
        pretrained=args.pretrained,
        epochs=args.epochs,
        batch_size=args.batch_size,
        lr=args.lr,
        weight_decay=args.weight_decay,
        optimizer=args.optimizer,
        scheduler=args.scheduler,
        seed=args.seed,
        deterministic=args.deterministic,
        device=args.device,
        amp=args.amp,
        resume=args.resume,
    )

    save_run_config(run_dir, cfg, argv)
    train(cfg, run_dir)
    return 0


if __name__ == "__main__":
    raise SystemExit(main(sys.argv[1:]))
```

8 Config File Example

Example file: `configs/baseline.json`

```
{
  "dataset": "rsna_trauma",
  "data_dir": "/datasets/rsna",
  "model": "resnet50",
  "epochs": 30,
  "batch_size": 16,
  "lr": 0.0001,
  "optimizer": "adamw",
  "scheduler": "cosine",
  "seed": 0,
  "amp": true
}
```

9 Checklist for Code Reviews

Before merging new training pipelines into the lab repo, ensure:

- Training can run fully from CLI with no code edits
- `--config` works and CLI overrides it
- Every run saves `config.json` and `cmd.txt`
- Output directory structure matches lab standard
- Seed is set and reproducibility toggles exist
- Dataset/model/optimizer use `choices` where applicable

10 Recommended Extensions (Optional)

- Add W&B or MLflow logging of the full config
- Add validation that checks incompatible arguments
- Add support for YAML configs (OmegaConf)
- Upgrade to Hydra for hierarchical config management